

Start with Medium. Make it earn the upgrade.

30:41

MEDIUM / SCOPEWELL

A checked workflow. A usable plan.

Mark Kashef | Seven first attempts, practical checks and copy-ready prompts.

Medium is my starting pick for this research-and-build brief. It returned a checked workflow and a useful plan in 30:41. High is the next setting I would test for a job with interacting constraints.

This guide gives you the evidence behind that call and a way to find the right setting for your own work. It includes the original assignment, seven editable canvases, ten reusable prompts and a run log.

Research locked 8 September 2026. The experiments ran on 7 September.

Choose the next setting for a reason.

1. Define done before choosing effort. Name the artifact you need and three checks that would make it useful. "Build an app" gives you much less signal than "change a price, approve it, refresh and retain the amount."
2. Start with Medium for a bounded mixed research/build project like this one. Start lower for an easy, tightly specified job if you can check the result quickly. These are practical starting hypotheses, not benchmark guarantees.
3. Escalate a specific failure. Try High when interacting constraints, a hard defect or a weak reasoning chain survives a targeted clarification. A missing screenshot alone is not a reason to increase every setting.
4. Compare the whole job. Include follow-ups, verification, waiting and child-agent work. Stop when the result passes your checks; do not chase a larger canvas or a more elaborate explanation.
5. Keep speed separate. The experiment used Standard speed. Mark's personal preference for Medium + Fast is a separate judgment, not an outcome tested by this batch.

The seven-run scoreboard.

SETTING	TIME	PROCESSED	SOURCES
Low	37:37	14.81M	8R + 0X
Medium	30:41	10.23M	6R + 0X
High	41:21	14.33M	7R + 1X
XHigh	45:14	12.29M	6R + 1X
Max	46:10	10.55M	7R + 0X
Sol High	32:49	6.70M	6R + 2X
Ultra	42:10	21.73M	7R + 2X

The table records these preserved first attempts, not a universal speed ranking. All seven ran concurrently on one machine. Ultra could delegate; the other arms could not.

Processed tokens = cumulative input + output. Cached input is already inside input. Most input in these runs was cached. Do not turn these totals into a price comparison.

None of the seven required a producer nudge to return artifacts. That is the result for these instructions and tasks. It does not establish whether higher effort cures early stopping.

Astra Low Clearshift

01 / THE ASTRA EXPERIMENT

Astra Low

CLEARSHIFT

Cleaning correction handoffs

TIME

37:37

elapsed

TOKENS PROCESSED

14.81M

98.3% of input cached

SOURCE THREADS

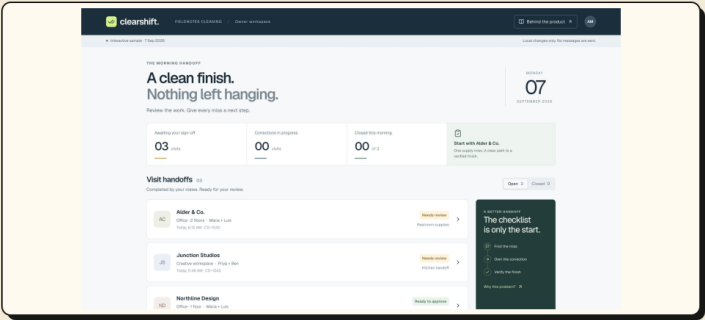
8

8 Reddit

CANVAS ELEMENTS

161

native, editable



Clearshift / actual prototype

PROVED

Required a recheck before closing

One run · identical assignment · shared machine

GAP

No X evidence; more work than Medium

Tokens include cached input

WHAT

Tracks missed cleaning tasks through correction and recheck.

FOR

Commercial cleaning owners managing recurring office sites.

PROPOSED MOAT

Client-specific standards and a history of fixes that worked.

APPROACH

Compared three service problems
Built a correction + recheck loop

DELIVERED

3-screen prototype
6-zone canvas · 3 app images
Sources, alternatives, validation plan

Tracks missed cleaning tasks through correction and recheck.

For: Commercial cleaning owners managing recurring office sites.

Proposed moat: Client-specific standards and a history of fixes that worked.

Observed check: A correction had to be rechecked before the job could close. The closed state survived a refresh.

Gap: No X evidence. More elapsed time and processed tokens than Medium.

What I would take from it: Use this as a reminder that Low can still produce a complete workflow. Lower effort does not guarantee a smaller total job.

Astra Medium Scopewell

02 / THE ASTRA EXPERIMENT

Astra Medium

SCOPEWELL

Scope changes for cleaning owners

TIME

30:41

elapsed

TOKENS PROCESSED

10.23M

98.0% of input cached

SOURCE THREADS

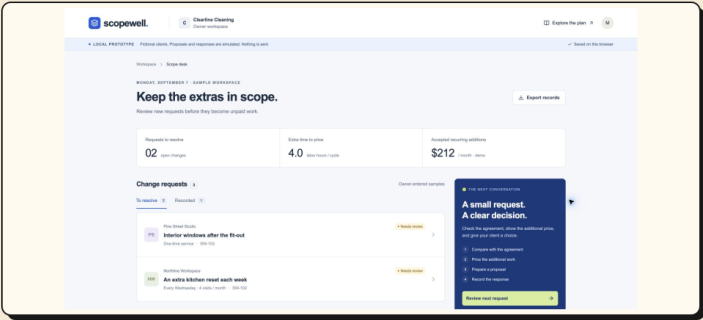
6

6 Reddit

CANVAS ELEMENTS

133

native, editable



Scopewell / actual prototype

WHAT

Prices extra cleaning work and records client acceptance.

FOR

Commercial cleaning owners with written service agreements.

PROPOSED MOAT

Accurate scope records and estimates improved by actual jobs.

APPROACH

Chose a narrow extra-work problem
Connected pricing to a proposal

DELIVERED

3-screen prototype
6-zone canvas · 1 wireframe image
Sources, alternatives, validation plan

PROVED

Changed price and approval survived reload

One run · identical assignment · shared machine

GAP

Lighter canvas; no X evidence

Tokens include cached input

Prices extra cleaning work and records client acceptance.

For: Commercial cleaning owners with written service agreements.

Proposed moat: Accurate scope records and estimates improved by actual jobs.

Observed check: Changing labor from 45 to 60 minutes changed the monthly proposal from \$164 to \$212. A simulated acceptance and the new amount survived refresh.

Gap: No X evidence. The lightest canvas by element count.

What I would take from it: My starting pick for this brief. It delivered a checked workflow and a useful plan in the shortest observed time.

Astra High Fieldwork

03 / THE ASTRA EXPERIMENT

Astra High

FIELDWORK

Recover a disrupted cleaning schedule

TIME

41:21

elapsed

TOKENS PROCESSED

14.33M

98.1% of input cached

SOURCE THREADS

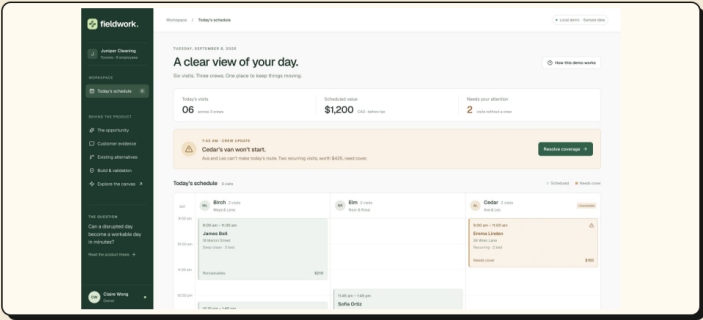
8

7 Reddit + 1 X

CANVAS ELEMENTS

186

native, editable



Fieldwork / actual prototype

WHAT

Reassigns visits when a cleaning crew cannot work.

FOR

Residential cleaning owners juggling crews and appointments.

PROPOSED MOAT

Company-specific constraints and outcomes that improve scheduling.

APPROACH

Compared service-owner problems
Built around scheduling constraints

DELIVERED

3-screen prototype
6-zone canvas · 3 app images
Sources, alternatives, validation plan

PROVED

Rejected an impossible time window

One run · identical assignment · shared machine

GAP

Travel uses a fixed allowance

Tokens include cached input

Reassigns visits when a cleaning crew cannot work.

For: Residential cleaning owners juggling crews and appointments.

Proposed moat: Company-specific constraints and outcomes that improve scheduling.

Observed check: An impossible two-hour job inside a one-hour window was rejected. Valid reassignment saved correctly and left four other visits untouched.

Gap: Travel used a fixed allowance, not live route optimization.

What I would take from it: A useful next setting to test when your job has interacting constraints. The value here came from the behavior it handled.

Astra Extra High Scopekeep

04 / THE ASTRA EXPERIMENT

Astra Extra High

SCOPEKEEP

Approve extra work, then prepare billing

TIME

45:14

elapsed

TOKENS PROCESSED

12.29M

97.8% of input cached

SOURCE THREADS

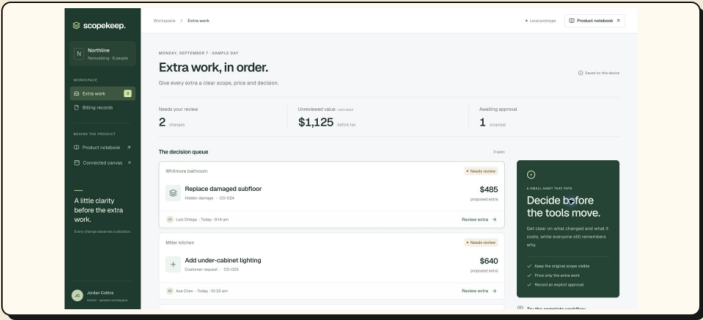
7

6 Reddit + 1 X

CANVAS ELEMENTS

155

native, editable



Scopekeep / actual prototype

PROVED

Repriced \$530 approval reached billing

GAP

45-minute allowance slightly exceeded

One run · identical assignment · shared machine

Tokens include cached input

Takes remodeling extras from approval into billing.

For: Small remodeling owners losing track of chargeable extras.

Proposed moat: Tailored scope checks informed by what got approved, billed and paid.

Observed check: A new labor rate flowed through a \$530 total, approval and a ready-to-bill row. Revision 1 survived refresh.

Gap: More elapsed time than High. Its X counterexample was participant-read; the producer could not independently reopen it.

What I would take from it: Read counterevidence before buying the business idea. An existing Grok plus open-source accounting workaround challenged the proposed product.

Astra Max Fieldnote

05 / THE ASTRA EXPERIMENT

Astra Max

FIELDNOTE

Keep scope, price and approval together

TIME

46:10

elapsed

TOKENS PROCESSED

10.55M

97.5% of input cached

SOURCE THREADS

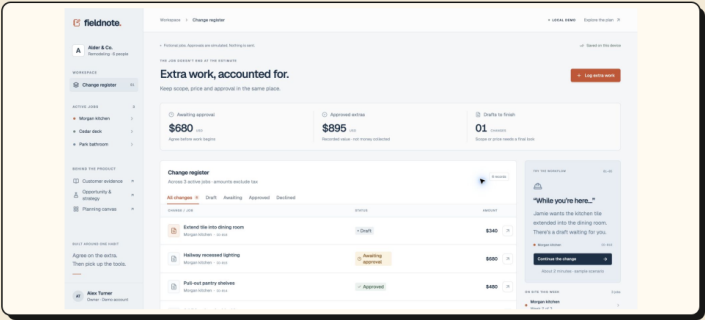
7

7 Reddit

CANVAS ELEMENTS

155

native, editable



Fieldnote / actual prototype

PROVED

\$360 approval persisted correctly

One run · identical assignment · shared machine

GAP

Native download remained unverified

Tokens include cached input

WHAT

Keeps an extra job's scope, price and approval in one record.

FOR

Residential remodeler owners with teams of 2–10 people.

PROPOSED MOAT

An adopted job-site routine, assisted setup and bookkeeper referrals.

APPROACH

Built an explicit change-record lifecycle
Checked cents, revisions and export

DELIVERED

3-screen prototype + text export
6-zone canvas · 3 app images
Sources, alternatives, validation plan

Keeps an extra job's scope, price and approval in one record.

For: Residential remodeler owners with teams of 2–10 people.

Proposed moat: An adopted job-site routine, assisted setup and bookkeeper referrals.

Observed check: Labor of \$275.25 plus materials of \$84.75 became exactly \$360. The simulated approval persisted.

Gap: Longest run. A native export download was not independently verified.

What I would take from it: Useful depth showed up in exact cents, approval state and revision rules. These are better checks than counting UI polish.

Sol High RelayOps

06 / THE ASTRA EXPERIMENT

Sol High

RELAYOPS

Recover daily field-service exceptions

TIME

32:49

elapsed

TOKENS PROCESSED

6.70M

97.5% of input cached

SOURCE THREADS

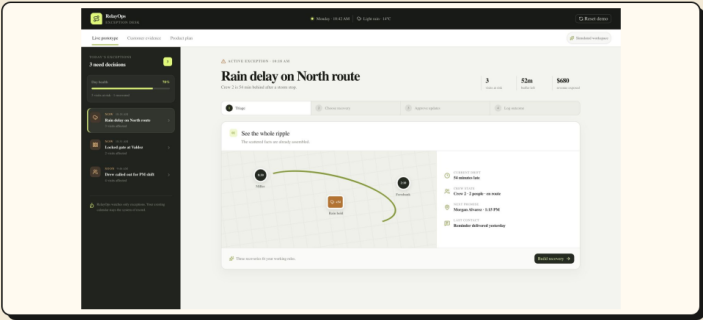
8

6 Reddit + 2 X

CANVAS ELEMENTS

215

native, editable



RelayOps / actual prototype

PROVED

Resolved state survived reload

GAP

Chosen option did not drive the summary

One run · identical assignment · shared machine

Tokens include cached input

WHAT

Plans recovery when delays disrupt a service route.

FOR

Owner-dispatchers of recurring service firms with 2–20 staff.

PROPOSED MOAT

Recovery outcomes, maintained rules and accountable support.

APPROACH

Picked a desk above existing calendars
Built a four-step recovery interface

DELIVERED

3-screen prototype
Canvas with vector screen diagrams
Source ledger + product-plan panels

Plans recovery when delays disrupt a service route.

For: Owner-dispatchers of recurring service firms with 2–20 staff.

Proposed moat: Recovery outcomes, maintained rules and accountable support.

Observed check: Step progression and a resolved state persisted.

Gap: Selecting Split with Crew 1 did not change the next summary: it still showed a different plan and fixed outcome numbers.

What I would take from it: Keep Sol in your comparison, then test whether choices actually change results. Low processed usage does not compensate for a broken decision path.

Astra Ultra

Accord

07 / THE ASTRA EXPERIMENT

Astra Ultra

ACCORD

Agree remodeling additions before work

TIME

42:10

elapsed

TOKENS PROCESSED

21.73M

97.3% of input cached

SOURCE THREADS

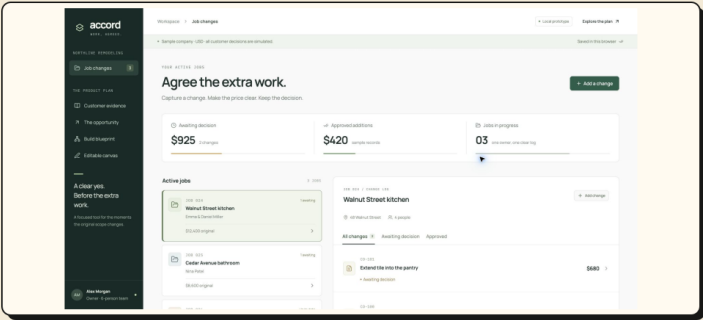
9

7 Reddit + 2 X

CANVAS ELEMENTS

240

native, editable



Accord / actual prototype

WHAT

Records and agrees remodeling additions before work starts.

FOR

Small residential remodeling owners handling frequent extras.

PROPOSED MOAT

Trade-specific setup and a proven routine, improved by real outcomes.

APPROACH

3 subagents: Reddit, X, alternatives
Reused specialists to check the build

DELIVERED

3-screen prototype + record export
6-zone canvas · 2 embedded images
Detailed competitor workflow research

PROVED

New \$200 record + approval persisted

One run · identical assignment · shared machine

GAP

3 subagents added 7.12M tokens

Tokens include cached input + all 3 subagents

Records and agrees remodeling additions before work starts.

For: Small residential remodeling owners handling frequent extras.

Proposed moat: Trade-specific setup and a proven routine, improved by real outcomes.

Observed check: A newly created custom \$200 change required an approval note. Its title, scope, total and approval persisted.

Gap: Three child agents added 7.12M processed tokens. This is a separate delegation-enabled condition.

What I would take from it: Use delegation when independent research or checking lanes have real value. Here it produced deeper competitor workflow research, not a dramatically different product.

A pretty result can hide a broken choice.

Sol's prototype advanced through the steps and retained a resolved state. That could have looked like a pass in a quick demo.

The producer chose "Split with Crew 1," but the following summary still described another plan. The outcome also stayed fixed. The selected input did not drive the downstream result.

Test cause and effect: choose a different option, change a number, use a new record, then check the exact consequence. Refresh afterwards. Also check that unrelated records did not change.

This single check is more useful than counting components, screens or canvas shapes. Use it on any model's build.

Review the finished prototype without editing its source first. Use a fresh or reset sample state and record the starting state.

1. Change a meaningful input or select a different option.
2. Predict which total, record or downstream summary should change.
3. Complete the main workflow.
4. Compare the final output with the choice actually made.
5. Refresh and verify persistence.
6. Check one invalid input and one unrelated record.

Capture receipts for each result. Separate tested behavior, source-code inspection and unverified behavior. Preserve the first-attempt artifact before proposing fixes. A working button or attractive screenshot is not sufficient evidence that the decision flows through.

Separate effort from follow-through.

Effort changes how much reasoning the model can put into a task. An incomplete scope, unclear authority or a real tool blocker can still stop the job. Turning the dial up does not fix every kind of pause.

Give it an observable finish line and the reversible decisions it can make. If you want research plus implementation plus verification, say that. Specify which actions still require your decision.

In this batch all seven returned artifacts without a producer nudge. We cannot rank “laziness” from a set where the measured nudge count did not differ.

Test autonomy separately: preserve the first attempt, count missing requirements and distinguish genuine blockers from avoidable early stops. Apply the same neutral follow-up policy to each run.

Complete this task: [TASK].

Done means: [OBSERVABLE DELIVERABLES AND ACCEPTANCE CHECKS].

You may decide [REVERSIBLE CHOICES] without asking me. Use reasonable assumptions where they do not change the objective; state any material assumption briefly.

Continue through implementation and verification within this scope. A plan, acknowledgement or offer to continue is not the finished deliverable. If you hit a real blocker, identify it, preserve the work and finish any independent parts that remain possible. Ask before [SPECIFIC ACTIONS REQUIRING MY DECISION].

At the end, show what you changed, the checks you actually ran and any requirement still unmet. Do not claim a test passed unless you ran it.

Run your own comparison cleanly.

Use the same frozen task text, starter files, available tools, sign-ins, speed setting and completion criteria. Keep every run in a fresh folder. Do not leak one model's output into another model's context.

Check the actual model and reasoning setting. A task named "High" is only a label. If your application cannot create or configure tasks, open and configure them manually.

For a fairer timing test, run one at a time or isolate their execution resources. Shared computer control can make simultaneous tests interfere. Repeat in a different order before making a strong claim.

Archive first attempts before repairs. Score the result without its effort label if possible. Preserve failure as evidence; do not quietly help your preferred setting.

Create three separate tasks for a comparison: "LOW | My test", "MED | My test" and "HIGH | My test". Use GPT-6 Astra at low, medium and high reasoning effort respectively, if supported here. Keep the speed setting identical. Verify and report the effective model and effort; labels alone are not enough.

Give all three the identical assignment below in fresh isolated folders. Do not include other runs or their outputs. Keep discretionary delegation disabled. Give each up to [TIME LIMIT] for a first attempt. Preserve its output before any follow-up. Record completed work, missing requirements, blockers, elapsed time and attributable usage when available.

Run in parallel only if their tools and workspaces are isolated; serialize any shared computer-control workflow. Monitor progress without coaching. If this environment cannot create tasks or set their effort, tell me precisely which step I must do manually. Never silently substitute settings.

ASSIGNMENT:

[PASTE ONE FROZEN ASSIGNMENT]

Millions of tokens are not a bill.

The model can see the same cached context across many calls. The cumulative processed count adds those reads. A 20M total does not mean one 20M-token context window or 20M freshly generated tokens.

For this dataset, total = input + output. Cached input is a subset of input; reasoning output is a subset of output. Adding those subsets again would double-count work.

Ultra's 21,730,368 includes the parent and three child agents. The children contributed 7,121,868. Comparing only the parent would hide part of the job.

A real API calculation needs uncached input, cached input and output rates for the specific model, speed and context tier. A Codex subscription uses its own credit rules. An account-wide allowance change cannot isolate a single task when other tasks are running.

The full ledger is in `evidence/results.csv` and `results.json`. Keep unavailable cost fields blank instead of guessing.

Make the research argue against the idea.

“What are the best nuggets?” often produces a polished summary. A narrower question can find a useful contradiction: who already solves this with an existing tool, refuses to pay, or says the problem is different?

In Extra High’s research, an owner’s existing Grok plus open-source accounting workflow challenged the need for a new product. That was useful because it changed the business argument. The producer could not independently reopen the X post, so the source remains participant-read.

Use Grok or another search route to find direct posts, then inspect the original evidence and missing context. Ask what task failed, what setting was used and whether anyone tested the suggested fix.

An anecdote can expose a failure mode worth testing. It cannot establish how often the model fails or which effort level caused it.

Help investigate this specific claim: [CLAIM]. Search X for direct experiences that could support or contradict it. Prioritize concrete task descriptions, screenshots with context, links to original posts and follow-up corrections.

For each finding report the original post URL, date, task, setting if stated, observed outcome and missing context. Separate the post author's opinion from verified product behavior. Do not turn likes, reposts or confident phrasing into proof.

Then give three narrower follow-up questions that would help explain why users got different results. If you cannot access the original evidence, say so.

A moat has to survive the copy.

The prompt explicitly asked what a customer or competitor could copy in a weekend, what would still be missing and why someone would pay. That is a harder question than asking for a feature list.

All seven products stayed in field services. The shared customer was a small service-business owner with 2–20 employees, which already narrowed the search. Convergence does not prove trades are the best market for everyone.

Stored records, a nicer interface and AI-generated templates are usually easy to describe as moats. The useful question is whether a real operating habit, permissioned outcome history, distribution or trusted setup service creates enough value that customers stay.

Every moat in these outputs is proposed. None of the prototypes had demonstrated paid adoption or a durable advantage. Set a small disproof test before building the larger system.

Assume a capable competitor and my customer can reproduce the software interface and basic logic in a weekend. Stress-test this product: [PRODUCT].

Separate what is easily copied from any advantage that has to be earned. Explain the day-one customer value before a moat exists. Identify how we could earn the first advantage from zero, why an incumbent could still beat us and what a customer might use instead.

Design a small first-customer experiment with an explicit pass/fail threshold. Label proposed defensibility as a hypothesis. Do not call a generic database, AI wrapper or feature list a proven moat.

272K is a per-request boundary.

For Astra API prompts above 272,000 input tokens, the whole request uses 2× input and cache rates and 1.5× output rates. It is not only the tokens above the line, and not every rate doubles.

A task can process millions of tokens across repeated calls without any one call crossing this boundary. Look at the largest individual input count, not the total printed at the end of the project.

This API boundary does not establish an equivalent Codex subscription discount. Before changing a TOML setting, check the installed client, sign-in method, supported setting and current value. Compaction can reduce retained detail too.

Source: OpenAI Astra model pricing notes, checked 8 September 2026.

Audit this workflow's context and billing settings without changing them. Identify whether it uses ChatGPT sign-in or API billing. For API calls, report the largest individual input-token count and whether any request exceeded 272,000 input tokens. Separate it from cumulative task tokens. If a config change would help, show the supported setting, current value and proposed diff first.

Fast costs more. Effort is another dial.

With ChatGPT sign-in, Astra Fast consumes credits at $2.5\times$ Standard's rate where available. That is a credit multiplier. It does not promise a task will finish $2.5\times$ faster.

The API has a separate pricing structure: Astra Fast uses $2\times$ applicable API token rates. Do not apply the subscription multiplier to an API bill.

Keep reasoning effort fixed when testing speed. Try one representative task on Standard and Fast, measure the experience and use actual attributable billing data if available. The seven-run experiment used Standard and did not measure this tradeoff.

In Codex CLI, `/fast status` inspects the setting; `/fast off` and `/fast on` change it. Confirm the corresponding setting in your own application.

Sources: Codex speed documentation and Astra API model notes, checked 8 September 2026.

Inspect my model, reasoning effort, speed mode and sign-in method. Explain the applicable usage tradeoff from current official documentation. Keep reasoning effort unchanged and show how to switch only speed so I can compare the same task on Standard and Fast.

Raise effort for the hard phase.

Astra supports a `configuration_update` input item to change reasoning effort while preserving the original request-level setting and cached prefix. You can draft at low, then raise effort for a difficult review in the same conversation.

Place the item before the next user message, while keeping the request-level `reasoning.effort` at its original value. The update persists until overridden. After compaction, add a fresh desired update.

This is a developer feature for Astra in standard single-agent mode, not a magic sentence that changes a Codex chat setting. The response's `reasoning.effort` still reports the request-level setting, so that field alone does not prove effective effort.

Shared history is useful for production work. It is not an independent clean-room comparison between effort levels.

Source: OpenAI, Change reasoning mid-conversation, checked 8 September 2026.

```
{
  "type": "configuration_update",
  "reasoning": { "effort": "high" }
}
```

Then ask for a specific difficult review, such as:

Review this migration for data loss, concurrency failures and rollback gaps. Point to the relevant part of the plan for each material risk. Propose a concrete check or change within the agreed scope.

Ask what stopped it. Bound what you delegate.

OpenAI's Astra guide describes sensitivity to skills and AGENTS.md instructions. When an unnecessary pause happens, ask which exact instruction or missing fact caused it. That gives you something concrete to fix.

If the task would benefit from delegation, give each agent a different bounded question. Customer evidence, existing alternatives and the strongest objections are useful separate lanes. The parent keeps the decision and verifies the integrated result.

Ultra's three specialists added 7.12M processed tokens in this run. Record their models and usage. More agents are not automatically cheaper and duplicate discovery can eat the expected benefit.

Sources: OpenAI Astra instruction-following and subagent-delegation guidance; preserved Ultra producer review.

If you are blocked, identify the exact missing fact or exact instruction and file causing the stop. Finish independent authorized work first.

If parallel work helps, use up to three agents with distinct questions and direct-source deliverables. Avoid duplicate searches. Keep synthesis and final verification with the main agent. If there is no useful independent work, continue without delegation.

One easy task. Three identical answers.

We also preserved the three small tasks used to demonstrate launching comparison chats during recording. Low, Medium and High received the same no-tools arithmetic question. All three returned the correct \$8 answer in the requested two-line format.

Low's recorded turn took 4,641 ms; Medium 5,461 ms; High 5,130 ms. They ran concurrently with starts within one second. These small differences are scheduling-sensitive and do not establish a reliable speed ranking.

For this one easy problem, higher effort added no visible answer or formatting benefit. It is a useful reason to test lower effort when the task is simple and the answer is easy to verify.

These three short demonstrations are separate from the seven research-and-build runs. Token usage was unavailable in the retrieved summaries. Full prompt, answer and data are in `evidence/BONUS-ARITHMETIC-DEMO.md` and `.json`.

Complete this simple task without tools: A notebook costs \$4 and a pen costs \$2. Maya buys 3 notebooks and 5 pens, then pays with \$30. How much change should she receive? Reply with exactly two lines: the calculation, then the answer.

All three answers:

$\$30 - (3 \times \$4 + 5 \times \$2) = \8

Maya should receive \$8 in change.

The exact task from the video.

This is the unchanged model-facing assignment from the video. Execution conditions and the annotated context board are separate files in experiment/.

Find a worthwhile SaaS opportunity for owners of small service businesses with 2-20 employees. I haven't chosen the business category, problem or product. Start with their problems, consider three opportunities, then choose one.

Research real conversations on Reddit and X and examine three existing alternatives. Aim to support the shortlist with six distinct, relevant conversations across those platforms. Look for repeated frustrations, current workarounds, spending signals and counterevidence. Choose your own research methods using the capabilities available to you. Link the original sources. Explain access limitations and evidence gaps; do not invent sources to meet a count. Popularity alone is not proof that someone will pay.

Assume customers and competitors can use Astra to reproduce competent software quickly. Explain what could be copied in a weekend, what valuable part would still be missing, and why a customer would pay instead of building or switching. Propose a credible way to earn that advantage from zero, including a first-customer strategy and an experiment that could disprove it. The product must deliver value before this advantage exists. Treat defensibility as a hypothesis, not an established moat.

Create an editable Excalidraw canvas with distinct, connected zones for customer evidence, alternatives and opportunity, the main user journey, product screens and visual direction, architecture and data flow, and MVP scope with build sequence. Use a useful combination of images, diagrams and concise annotations, with source links.

Build a polished website that combines the explorable product plan and working prototype in one experience. Build for one primary user role and one core problem. Implement one complete workflow with 3-5 meaningful steps and at most three main product screens. Use realistic sample data and preserve changes across a refresh. Keep research and planning material in supporting panels. Exclude production authentication, payments, live messaging and external integrations; identify simulated behavior clearly.

Deliver the working prototype, editable Excalidraw canvas, planning website and research sources. Verify the main workflow and explain what remains unproven. Make the product and implementation decisions within this scope. Do not contact people, post publicly, purchase services or deploy publicly. Do not claim research or verification you did not perform.

Keep the receipt. Then improve it.

Choose a real task you already need to finish. Write three pass/fail checks. Run Medium once, keep the result and test it. If it misses a check, try one targeted follow-up before deciding that more effort was the missing ingredient.

For a comparison, use the blank CSV in `experiment/YOUR-RUN-LOG.csv`. Record effective settings, speed, delegation, first-attempt results and follow-ups separately. Keep screenshots or output files next to each row.

Open the original canvases in Excalidraw to inspect their structure and sources. Use the seven scorecard images as a quick visual reference. Everything essential in this companion works offline.

Video companion: [Open the interactive guide](#)

You are choosing a setting that earns its place in your workflow. Start with the useful result. Let the failure tell you what to change.

Sources and measurement.

Official product facts were checked on 8 September 2026. SOURCE-NOTES.md contains the fuller research, use cases and source links. Product behavior, availability and prices can change.

Experiment data: seven preserved first attempts from 7 September. Each run's cumulative input and output were independently reconciled. Duration came from the first completion event. The shared machine and different product choices limit causal timing and effort conclusions.

Producer checks: the actual interaction paths described in evidence/PRODUCER-CHECKS.md. These checks are deeper inspection of the original seven outputs, not additional independent model runs.

The guide's task-setting suggestions and reusable prompts are practical recommendations. Paid adoption, product moats and universal superiority were not established by this experiment.

[Astra prompting: instruction following and delegation](#)

[Astra model and API pricing notes](#)

[Codex speed and credit usage](#)

[Responses API: changing reasoning mid-conversation](#)

[Prompt caching: measurement and conditions](#)

[Computer use: continue and verify the result](#)

[Tibo's comparison post](#)

[Video's interactive companion](#)